第 2 章 • 预备知识 / 2.7

2.7 查阅文档

把陌生函数的说明，变成能够验证和解释的调用。

发现名字 → 读懂签名 → 最小验证 → 查阅与实践

本节目标

能够用 dir 和 help 查找工具，读懂基本签名、参数和返回值，核对版本，并通过最小实验完成一个陌生函数的调用。

建议先连续阅读完整知识链，再运行示例，最后集中完成章末练习。已会的内容可快速通过，重点追踪自己无法解释的结果。

教材定位：PDF 第 99-101 页 / 正文第 81-83 页。

参考《动手学深度学习》Release 2.0.0（2023-08-18）及本项目已验证讲义整理。此 PDF 为独立学习读本，配套程序与练习均在项目内。

这节要建立的习惯

不需要先记住 PyTorch 的全部函数。遇到任务时，先说清输入和目标，再找到候选函数，读它接受什么、返回什么，最后用很小的数据验证。查文档是在确认一个“输入到输出的约定”。

例如目标是“每条样本内部求和，并保留一列以便后续广播”。这比只问“sum 怎么用”更具体：输入是二维张量，输出应为 (样本数,1)。

一轮查询	要回答的问题
找名字	这个模块中有什么候选工具?
看签名	输入、参数、默认值是什么?
读返回值	返回张量、多个值，还是其他对象?
做小实验	数值、shape、dtype 与预测一致吗?
用回任务	是否满足原来提出的目标?

从模块找到名字，再定位帮助对象

教材用 dir 查看 torch.distributions。dir 返回对象的属性名列表，其中可能有函数、类、子模块和数据属性，并非每一项都能作为函数调用，也不是保证完整的 API 清单。

```python
import torch

names = dir(torch.distributions)
public_names = [name for name in names
                if not name.startswith("_")]
print("Multinomial" in public_names)
```

列表推导式可以读成：“遍历 names 中的每个 name，只留下不以下划线开头的名字”。单下划线常表示内部使用，双下划线常是 Python 特殊协议；这里只为减少查找时的干扰，并非断言它们无用。

找到了名字，再查询具体用法。help 会把帮助内容打印出来，它本身返回 None。

```python
help(torch.ones)
```

这一行没有写 torch.ones()。函数本身也是一个对象，可以作为 help 的参数。加上括号表示先执行调用，再把得到的结果交给 help；torch.ones() 没有提供形状，还会先发生调用错误。

类似地，help(torch.ones(2,3)) 查的是调用得到的张量对象，不是 ones 函数的参数说明。实际操作时直接写 help(torch.ones)，不用再套 print，否则末尾会多打印一个 None。

看清点号路径

写法	目前怎样理解
torch	导入的 PyTorch 顶层模块
torch.distributions	包含概率分布工具的子模块
torch.distributions.Multinomial	一个分布类
torch.ones	一个创建张量的函数对象
x.sum	张量对象 x 上的求和方法

函数调用常写 torch.sum(x,...)；对应方法写 x.sum(...)，这两个求和入口在本节参数下结果一致。点号表示访问对象的属性。

把签名读成一次真实调用

下面是本机 torch.ones 的签名，已为阅读换行；它描述调用格式，无需复制成 Python 函数定义。

结果 / 结构

```
ones(*size, *, out=None, dtype=None,
     layout=torch.strided, device=None,
     requires_grad=False) -> Tensor
```

文档记号	这里的含义
*size	可以传多个尺寸，如 2,3；也可传一个形状元组
单独的 *	标记后面的选项需要写参数名，如 dtype=...
参数=默认值	不提供该参数时采用的设置
None	Python 的空值；具体含义由这个参数的说明决定
-> Tensor	返回一个张量，不是实际需要输入的符号
optional	可以省略；具体默认行为仍要读说明

某些函数签名还会出现 /，表示它前面的参数只能按位置传递。先识别它，不需要现在背诵所有签名规则。

PYTHON

```
a = torch.ones(2, 3, dtype=torch.float64,
               device="cpu", requires_grad=True)
b = torch.ones((2, 3), dtype=torch.float64)
print(a.shape, a.dtype, a.device, a.requires_grad)
print(torch.equal(a, b))
```

两种形状写法都得到 (2,3)，数值都是 1；a 开启梯度跟踪，b 使用默认的 requires_grad=False，因此数值相同不代表属性完全相同。torch.equal 在这里比较张量的形状和数值，不检查所有配置属性。

dtype 指定数据类型，device 指定存放设备，requires_grad 控制梯度跟踪。

out 指定接收结果张量，layout 说明存储布局。目前认识即可，按任务选择需要的选项。

用最小数据验证含义

沿用已经学过的求和，通过文档确认 dim 与 keepdim。torch.sum 有不同调用形式：只给 input 会把全部元素求和；带 dim 可以指定汇总轴。

PYTHON

```
x = torch.tensor([[2., 4., 6.],
                  [3., 6., 9.]], dtype=torch.float64)
total = torch.sum(x)
row_sums = torch.sum(x, dim=1)
kept_sums = torch.sum(x, dim=1, keepdim=True)
normalized = x / kept_sums

print(total, total.shape)
print(row_sums, row_sums.shape)
print(kept_sums, kept_sums.shape)
print(normalized.sum(dim=1))
```

表达式	数值与形状
total	30；标量，形状 ()
row_sums	[12,18]；形状 (2,)
kept_sums	[[12],[18]]；形状 (2,1)
normalized	两行都是 [1/6,1/3,1/2]；形状 (2,3)

keepdim=True 把被汇总的轴保留为长度 1，方便广播；它没有改变求和本身的数值。若直接写 x/row_sums，(2,3) 与 (2,) 的末轴不匹配，会报错。

完整演示故意捕获两种错误：漏掉保留轴导致的形状错误，以及误把 keepdim 写成 keep_dim 的参数错误。它们属于教学对照，程序会继续运行。

读报错时先看最后一条异常，找出函数名、参数名或不兼容的形状，再回到帮助中的对应位置核对。程序能运行后，还需核对结果含义；有些广播错误不会报异常。

在 VS Code 与在线文档中查阅

在 VS Code 的 Python 文件中，把鼠标停在函数名上通常能看到帮助提示；输入参数时可查看签名提示。这依赖已安装的 Python 语言支持和所选解释器。提示不足时，直接运行 help，或阅读本节生成的本机帮助文件。

PYTHON

```
print(torch.__version__)
print(torch.ones.__doc__.strip().splitlines()[0])
```

本机验证版本是 2.4.1+cu121。在线文档中的 stable 会随官方发布变化，不能把网页的版本当作本机版本；遇到差异，先核对本机 help 和最小实验，再选择相应版本的文档。

教材还介绍了 IPython/Jupyter 的 ? 和 ??。它们是交互环境扩展，不是普通 Python 语法，不应直接写入本项目 .py 文件。

结果 / 结构

```
# 仅在支持的 IPython/Jupyter 环境中输入
torch.ones?
torch.ones??
```

一个问号用于查看帮助；两个问号会尝试显示更详细的信息及可取得的源码。内置函数、C/C++ 扩展或没有可访问源码的对象，并不保证能显示 Python 实现。教材中的 list?? 也不能理解为一定能取到 list 的 Python 源码。当前项目使用 .py，不必为问号语法安装 Notebook。

本节在线参考（2026-10-19 查询，stable 是动态链接）：

- PyTorch ones：https://docs.pytorch.org/docs/stable/generated/torch.ones
- PyTorch max：https://docs.pytorch.org/docs/stable/generated/torch.max
- Python dir/help：https://docs.python.org/3.13/builtins/functions.html

上述链接用于进一步阅读；所有演示以本机 Python 3.8.20、PyTorch 2.4.1 的实际帮助与执行结果核对。完整本机帮助保存在 reports/ch02/2_7_local_help.txt，离线也可阅读。

运行入口

在项目根目录运行，或按 F5 选择“2.7 查阅文档：完整演示”。

结果 / 结构

```
& D:\Anaconda\envs\openmmlab\python.exe `
  .\lessons\ch02\07_documentation.py
```

一次完成综合练习

打开 exercises/ch02/07_documentation.py。三条样本各有四个类别的概率，任务是找出每条样本最高的概率及对应类别编号，两个输出都保留为 (3,1)。这里使用人工概率，不涉及模型训练。

```python
probabilities = torch.tensor([
    [0.10, 0.60, 0.20, 0.10],
    [0.15, 0.10, 0.55, 0.20],
    [0.70, 0.05, 0.15, 0.10],
], dtype=torch.float64)
```

集中完成三处 TODO：查 torch.max 的帮助，选择合适签名并调用；再从返回对象中分别读出最大值和索引。不要硬编码结果。阅读提示：同名函数可能有多个签名；带 dim 的返回值说明与不带 dim 的不同。

本节练习没有提前填答案。若文档写返回 values 与 indices，需要分清“分数本身”和“分数所在的位置”；索引从 0 开始，不能当作最大值。

集中反馈运行结果与三个解释：

1. 你选择了哪条签名？为什么只写 torch.max(probabilities) 不能完成每条样本的任务？
2. 最大值和索引各表示什么？keepdim=False 后，两个输出的形状怎样变化？
3. help(torch.max) 与 help(torch.max(probabilities)) 查询的是同一个对象吗？

阅读后的速查

遇到的问题	优先做什么
不知道模块里有什么	dir 后按任务筛选名字
不知道参数和返回值	help(函数本身)，读对应签名
看懂文字但不确定含义	用几行确定数据验证数值与 shape
示例来自其他版本	查看 torch.__version__ 和本机 help
想看实现	尝试源码查询，接受内置实现未必可见

本节很短，不需要背出模块的全部名字。能通过文档完成一次新函数调用，就达到了当前实践目标。下一节进入第 3 章 3.1 线性回归：把数据、模型、损失、梯度和参数更新接成完整训练问题。

资料说明：教材作者为 Aston Zhang、Zachary C. Lipton、Mu Li、Alexander J. Smola；页码基于用户提供的中文 PDF。文中相对路径均从“李沐-深度学习”项目根目录计算。已验证环境：Python 3.8.20、PyTorch 2.4.1+cu121、NumPy 1.23.5、pandas 2.0.3。整理日期：2026-10-19。